

TECHNICAL BRIEF · SELF-IMPROVING AD GENERATION

The Recommendation & Fine-Tuning *Engine*

How human ratings are converted into mathematics that steer scenario selection, prompt tuning, and video-script tuning — the methods, the formulas worked from first principles, and the reasoning behind each choice.

learn ingredients · not whole scenarios

Alluvi · UGC Pipeline

Statistical design & rationale

01 What the engine does

The pipeline already produces UGC-style ads: a persona, a product box, a scenario, an image, then a video. The engine adds one thing — it makes that output *get measurably better over time* by turning each human quality rating into a statistical signal.

It performs four jobs in sequence: **score** each rated asset into clean numbers; **learn** which production *ingredients* reliably succeed; **pick** which scenarios to generate next; and **tune** the image prompt and video script by converting repeated failures into validated edits.

CORE DESIGN DECISION

We learn about **ingredients (attributes)** — grip type, lighting, framing, fabric, scene — not whole scenarios. At 100+ videos/day every full scenario is nearly unique, but the same grip or lighting recurs across dozens of them. Learning at the ingredient level is the only way to accumulate enough samples to be statistically sound while keeping every video visually distinct.

02 The four layers

Each layer is deliberately separate because each uses different mathematics and serves a different goal.

L1

Per-asset scoring *weighted mean · AND-gate logic*

Turn one human rating into usable numbers: pass/fail fidelity gates and 1–5 quality scores.

L2

Attribute statistics *Beta-Bernoulli · Bayesian shrinkage*

Estimate how reliable each ingredient is, with honest handling of small samples.

L3

Selection *multi-armed bandit · Thompson sampling*

Choose the next scenarios — favouring what works while still exploring.

L4

Fine-tuning *credit assignment · A/B validation*

Diagnose failures and turn them into validated prompt and script edits.

STEP 1 — FROM A RATING TO NUMBERS

03 Scoring a single asset

A reviewer answers two kinds of question per asset: **gates** (objective pass/fail — is the colour right, is the grip plausible, is the brand text legible) and **scores** (1–5 quality — aesthetics, lighting, ad-worthiness).

Gates use logical AND. An asset is usable only if *every* gate passes; one failure rejects it. We never average gates, because a wrong product colour is not "offset" by nice lighting — fidelity is non-negotiable.

Scores are normalised, then averaged. Each 1–5 becomes 0–1 via $(s - 1) / 4$; we average only the answered ones (a blank is *missing*, never zero). We keep two composites: a **craft** score (look and feel) and a **commercial** score (ad-worthiness / hook).

WORKED EXAMPLE

A rated image scores aesthetic 3, detail 3, scene-adherence 4, ad-worthiness 4, lighting blank.

```
craft = mean( 4→0.75, 3→0.50, 3→0.50 )    # lighting blank → skipped
          = (0.75 + 0.50 + 0.50) / 3 = 0.583
commercial = ad-worthiness 4 → 0.75
```

One click becomes: `usable = true`, craft 0.58, commercial 0.75 — plus a record of which ingredients were present, ready for Layer 2.

STEP 2 — LEARNING RELIABILITY

04 How reliable is an ingredient?

A gate outcome is a *Bernoulli trial* — pass or fail. The naïve answer "passes ÷ total" is dangerously wrong on small samples: three passes out of three is not a 100%-reliable ingredient, you simply have not seen it fail yet.

The fix is the *Beta-Bernoulli conjugate model* (an application of *Bayes' theorem*). We give every ingredient a few imaginary results before counting real ones — the *prior*. With a weak prior of 2 imaginary passes and 2 imaginary fails (this is *Laplace / additive smoothing*):

```
estimate = (alpha0 + passes) / (alpha0 + beta0 + n)    # weak prior: alpha0 =
beta0 = 2
empirical-Bayes prior: alpha0 = s·p_bar, beta0 = s·(1 - p_bar)  # p_bar = global
pass-rate, s ≈ 10
```

WORKED EXAMPLE — THE GRIP GATE, BY GRIP TYPE

Grip ingredient	Raw	Calculation (prior 2/2)	Estimate
placed on surface	8/8 = 100%	$(2+8) / (4+8)$	0.83
two-handed cradle	4/5 = 80%	$(2+4) / (4+5)$	0.67
held with phone	2/6 = 33%	$(2+2) / (4+6)$	0.40

The illusory "100%" is pulled down to a believable **0.83**; the harsh "33%" is pulled up to **0.40**. The fewer the samples, the stronger this pull toward the mean — a behaviour called *shrinkage*.

WHY THIS METHOD

It refuses to over-react to thin evidence, yet as real samples accumulate the imaginary prior becomes negligible and the estimate converges to the true rate. It is the same principle credit-card and ad systems use to estimate rare-event rates from limited data.

Bayes' theorem

Beta-Bernoulli

Laplace smoothing

Empirical Bayes

STEP 3 — LEARNING QUALITY

05 How good does an ingredient look?

The same small-sample danger applies to the 1–5 quality scores. We use the *Bayesian average* — the exact formula IMDb uses to rank films, and an instance of the *James–Stein shrinkage estimator*. It blends an ingredient's own mean with the global mean, weighted by how much evidence it has:

```
Q_hat = (n · mean_attr + k · C) / (n + k)
# mean_attr = its own average · C = global mean · k = pseudo-count (≈ 10–20)
```

WORKED EXAMPLE — AESTHETIC SCORE, BY LIGHTING (C = 3.8, K = 5)

Lighting ingredient	Samples	Raw mean	Shrunk Q_hat
golden hour	n = 6	4.5	$(6 \cdot 4.5 + 5 \cdot 3.8) / 11 = 4.18$
warm/cool dual	n = 2	4.8	$(2 \cdot 4.8 + 5 \cdot 3.8) / 7 = 4.09$

Raw numbers say "warm/cool" (4.8) beats "golden hour" (4.5). After shrinkage the order **flips**: golden hour (4.18) wins, because its score was earned over six samples while warm/cool's higher number rests on only two. The system declines to trust a high average built on almost no evidence.

WHY THIS METHOD

A simple average lets a single lucky 5★ masquerade as the best ingredient. Shrinkage requires an ingredient to *earn* its ranking with repeated evidence — the rating literature treats roughly 30 samples as the threshold for full trust.

Bayesian average

IMDb weighted rating

James–Stein shrinkage

STEP 4 — QUANTIFYING UNCERTAINTY

06 Not just a guess — a belief

Each ingredient is stored not as one number but as a **belief distribution** that carries both a best guess and a level of certainty. For gates that belief is a *Beta distribution*; for scores, a *Normal distribution* whose spread narrows as samples grow.

```
gate belief: Beta(alpha0 + passes, beta0 + fails)
score belief: Normal( Q_hat , sd / sqrt(n + k) ) # spread shrinks with more data
```

A well-tested, good ingredient has a **narrow** belief sitting high; an untested one has a **wide** belief. That width is what powers intelligent exploration in the next step. (For conservative ranking we could also use the *Wilson score interval* — the method Reddit uses — but the Beta belief already gives us everything we need.)

All of this is stored compactly as *sufficient statistics* — running totals `n`, `passes`, `sum`, `sum-of-squares` — so each new rating is an $O(1)$ update, never a recomputation over history.

STEP 5 — CHOOSING WHAT TO MAKE NEXT

07 The selection engine

Picking the next scenario is the classic *exploration–exploitation trade-off*, formalised as the *multi-armed bandit problem*: keep using what works, but keep testing what might be better. We solve it with *Thompson sampling* (also called *posterior sampling*) — the approach ad platforms use for creative optimisation, which also handles delayed, batched feedback gracefully.

How a "draw" is computed

To choose, we draw one random sample from each ingredient's belief and pick the highest. A draw from a Beta belief is produced from two *Gamma* samples:

```
draw ~ Beta(a, b) = X / (X + Y), X ~ Gamma(a), Y ~ Gamma(b)
example Beta(10, 2): X = 9.1, Y = 1.5 → 9.1 / (9.1 + 1.5) = 0.86
```

WORKED EXAMPLE — SIX SELECTION ROUNDS ACROSS THREE GRIPS

Round	placed (sure, high)	cradle (mid)	phone (unsure, low)	winner
1	0.86	0.71	0.52	placed
2	0.74	0.81	0.38	cradle
3	0.90	0.66	0.61	placed
4	0.79	0.58	0.83	phone ← explore
5	0.84	0.77	0.44	placed
6	0.70	0.69	0.55	placed

The strong ingredient wins most (exploitation); the weak one still gets an occasional turn whenever its wide belief happens to draw high (exploration). The balance is automatic — no thresholds to tune.

Always pick the current best never discovers improvements and makes every video identical. *Pure random* wastes budget. *ϵ -greedy* explores blindly regardless of confidence. Thompson sampling explores in proportion to genuine uncertainty, which is both efficient and exactly what keeps the catalogue diverse instead of collapsing onto a few "safe" looks.

Two guardrails

- **Diversity filter (applied first):** remove any background + outfit combination used recently for that persona, then let the bandit choose among what remains — so the result is both high-performing and fresh.
- **Exploration floor:** a small fraction of picks are fully random, guaranteeing brand-new ingredients always enter the system.

Beliefs are conditioned on context (the persona's country, gender, age) — a *contextual bandit* — so an ingredient that works in one market is not assumed to work everywhere.

Multi-armed bandit

Thompson sampling

Exploration-exploitation

Gamma sampling

Contextual bandit

STEP 6 — FINE-TUNING PROMPTS & SCRIPTS

08 Turning failures into fixes

When an asset scores poorly, which part is to blame — the ingredient choice, or the wording of the prompt?

Assigning blame correctly is the *credit-assignment problem* from reinforcement learning. We handle it in two tracks so a wrong guess can never silently corrupt production.

Track	What happens	Safeguard
1 • Hypothesise	The failing frame, the prompt, and the rubric result are sent to the Claude API, which returns a structured guess: the likely cause and a suggested edit.	Stored as candidate — never auto-applied.
2 • Validate	The edit is A/B tested — generate with and without it — and the change in the target score is measured (<i>A/B testing</i>).	Promoted to validated only on a real improvement.

WORKED EXAMPLE — A TUNING SUGGESTION'S LIFE

```
# observed: vertical box holds fail the grip gate more often
cause : "vertical orientation makes the hand pose ambiguous"
edit  : "specify a thumb + two-finger pinch on the long edge"
status: candidate → testing → validated (grip-gate pass-rate +0.21)
```

Image-gate and image-score failures route their edits to the **image prompt**; motion, camera, and hook failures route to the **video script** — the same mechanism, selected by which rubric dimension failed.

An unverified "cause" is just a plausible story; only a measured score lift earns trust. And if a gate keeps failing *regardless* of wording, the problem is a model limitation, not a prompt bug — so we stop editing text and simply let Layer 3 select that ingredient less. (Brand-text legibility is the textbook example of a model-limit failure.)

Credit assignment

A/B testing

FOUNDATIONS

09 Reward hierarchy & lifecycle

What "good" means, in order of authority

We never collapse quality into one number, because fidelity and preference are different things.

- **Gates** — fidelity that must always hold (correct colour, plausible grip, legible text). Never traded off.
- **Craft** — the 1–5 human judgement of how good it looks.
- **Commercial** — ad-worthiness today, to be replaced by *real platform performance* (views, watch-through) once available. That is the true reward; the human score is its stand-in until the data exists.

How the engine switches on

For every tenant — even a brand-new one — the engine stays **off** until that tenant has run and rated its full set of curated scenarios. This deliberate *burn-in (forced-exploration) phase* guarantees the statistics are warm before any steering begins. Completion is measured robustly: a scenario counts as resolved once it reaches a terminal state — either rated, or skipped by quality-control — so a scenario the model can never render cannot trap a tenant below the finish line. When coverage is reached, the engine flips to **active** once, and selection and tuning come online.

WHY A BURN-IN FIRST

A bandit that exploits before it has explored locks onto whatever looked good first. Forcing the full curated set to run once gives every ingredient a fair initial sample, so the engine's first real decisions rest on evidence rather than luck.

10 Methods at a glance

Where	Method	Formal name	Why we use it
Per-asset	AND gates · normalised mean	—	Keep fidelity non-negotiable; treat blanks as missing.
Gate reliability	Smoothed pass-rate	<i>Beta-Bernoulli · Laplace · Empirical Bayes</i>	Honest estimates from small samples.
Quality scores	Shrunk average	<i>Bayesian average · James–Stein</i>	Don't trust a high mean built on few samples.
Uncertainty	Belief distributions	<i>Beta · Normal · Wilson</i>	Carry certainty, not just a point guess.
Selection	Pick by sampled belief	<i>Thompson sampling · MAB · contextual bandit</i>	Explore by uncertainty; protect diversity.
Tuning	Hypothesise then verify	<i>Credit assignment · A/B testing</i>	Only measured improvements reach production.
Storage	Running totals	<i>Sufficient statistics</i>	Constant-time updates, no history rescans.

In short: fidelity is enforced as hard gates, quality is estimated with shrinkage so thin evidence is never over-trusted, selection uses Thompson sampling so the system improves without going stale, and every prompt change is validated before it is trusted — with real ad performance as the eventual ground truth.